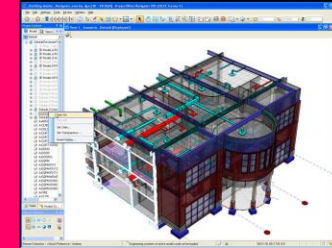


수요 예측 실습 및 사례 분석

문일경 교수

서울대학교 산업공학과

Construction

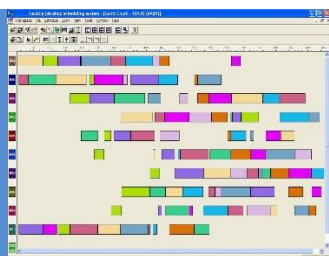


Operations
Research



Simulation

Scheduling



Analysis

Copyright © 2025 Supply Chain Management Lab
무단 복제 엄금



ENGINEERING
COLLEGE OF ENGINEERING
SEOUL NATIONAL UNIVERSITY

수요 예측 실습 및 사례 분석

[LG 실습 자료 다운로드](#)

<http://scm.snu.ac.kr/src/LG%EC%8B%A4%EC%8A%B5.zip>

Contents

0. 실습 개요
1. 데이터 분석
2. 회귀분석
3. 시계열 모델
4. 머신러닝 모델
5. 하이브리드 모델
6. 딥러닝 모델

0. 실습 개요

- 사례 분석 데이터 : 한국토지주택공사 구내식당 데이터
 - ✓ 일자, 요일별 조식/중식/석식 메뉴 및 식수 인원
 - ✓ 계절지수 및 선호지수 산출 가능

구내식당 식수 인원 예측 AI 경진대회

알고리즘 | 정형 | 회귀 | 행태심리 | MAE

₩ 상금 : 총 700 만원

🕒 2021.06.03 ~ 2021.07.23 18:07 [+ Google Calendar](#)

👤 2,417명 📅 마감

XP를 획득했어요!

[참여중](#)

[train.csv](#)
[test.csv](#)
[sample_submission.csv](#)

Views | Grid view | 4 hidden fields | Filter | Group | Sort

	일자	요일	본사정원수	조식메뉴	중식메뉴	석식메뉴	중식계	석식계
1	2016-02-01	월	2601	모닝롤/편빵 우유/두유/...	쌀밥/잡곡밥 (쌀,현미흑미:...	쌀밥/잡곡밥 (쌀,현미흑미:...	1039.0	331.0
2	2016-02-02	화	2601	모닝롤/단호박샌드 우유...	쌀밥/잡곡밥 (쌀,현미흑미:...	콩나물밥*양념장 (쌀,현미...	867.0	560.0
3	2016-02-03	수	2601	모닝롤/베이글 우유/두...	카레덮밥 (쌀,현미흑미:국...	쌀밥/잡곡밥 (쌀,현미흑미:...	1017.0	573.0
4	2016-02-04	목	2601	모닝롤/토마토샌드 우유...	쌀밥/잡곡밥 (쌀,현미흑미:...	미니김밥*겨자장 (쌀,현미...	978.0	525.0
5	2016-02-05	금	2601	모닝롤/와플 우유/두유/...	쌀밥/잡곡밥 (쌀,현미흑미:...	쌀밥/잡곡밥 (쌀,현미흑미:...	925.0	330.0

0. 실습 개요

- 사례 분석 데이터 : 한국토지주택공사 구내식당 데이터
 - 2016.2.1 ~ 2021.1.26 기간의 구내 식당 식수 인원 데이터

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

일자	요일	본사정원수	휴가자수	출장자수	재택근무자수	중식메뉴	중식계
2016-02-01	월	2601	50	150	0	쌀밥/잡곡밥 (쌀,현미흑미:국내산) 오징어찌개 쇠불고기 (쇠고기:호주산) 계란찜 청포묵무침 요구르트 포기김치 (배추,고추가루:국내산)	1039
2016-02-02	화	2601	50	173	0	쌀밥/잡곡밥 (쌀,현미흑미:국내산) 김치찌개 가자미튀김 모듬소세지구이 마늘종무침 요구르트 배추겉절이 (배추,고추가루:국내산)	867
2016-02-03	수	2601	56	180	0	카레덮밥 (쌀,현미흑미:국내산) 팽이장국 치킨핑거 (닭고기:국내산) 쫄면야채무침 견과류조림 요구르트 포기김치 (배추,고추가루:국내산)	1017
2016-02-04	목	2601	104	220	0	쌀밥/잡곡밥 (쌀,현미흑미:국내산) 쇠고기무국 주꾸미볶음 부추전 시금치나물 요구르트 포기김치 (배추,고추가루:국내산)	978
2016-02-05	금	2601	278	181	0	쌀밥/잡곡밥 (쌀,현미흑미:국내산) 떡국 돈육쌈양강정 (돼지고기:국내산) 우영잡채 청경채무침 요구르트 포기김치 (배추,고추가루:국내산)	925
2016-02-11	목	2601	383	143	0	쌀밥/잡곡밥 (쌀,현미흑미:국내산) 시래기국 훈제오리구이 도토리묵무침 쌈무/양파절임 요구르트 포기김치 (김치:국내산)	1045
2016-02-12	금	2601	389	156	0	쌀밥/잡곡밥 (쌀,현미흑미:국내산) 꽃게탕 돈육굴소스볶음 옥수수전 유채나물 요구르트 포기김치 (김치:국내산)	909
2016-02-15	월	2601	87	204	0	쌀밥/잡곡밥 (쌀:국내산) 시금치국 닭감자조림 (닭고기:국내산) 연두부*양념장 콩나물무침 요구르트 포기김치 (김치:국내산)	1268
2016-02-16	화	2601	72	236	0	쌀밥/잡곡밥 (쌀:국내산) 쇠고기무국 (쇠고기:호주산) 탕수어 (동태:러시아산) 오징어숙회무침 취나물 요구르트 포기김치 (김치:국내산)	1014
2016-02-17	수	2601	78	250	0	쌀밥/잡곡밥 (쌀:국내산) 냉이된장국 쇠고기장조림 (쇠고기:호주산) 통도라지구이 치커리무침 요구르트 포기김치 (김치:국내산)	916
2016-02-18	목	2601	77	254	0	낙지비빔밥 (낙지:중국산,쌀:국내산) 유부장국 고구마치즈구이 해초무침 요구르트 요구르트 포기김치 (김치:국내산)	910
2016-02-19	금	2601	145	252	0	쌀밥/잡곡밥 (쌀:국내산) 대구찌개 마늘간장치킨 (닭고기:국내산) 새송이버섯볶음 돌나물무침/치킨무 요구르트 포기김치 (김치:국내산)	763
2016-02-22	월	2601	90	207	0	쌀밥/오곡밥 (쌀:국내산) 쇠고기스타리국 갈치구이 호박고지&고사리&무나물 파래김*양념장 부림/요구르트 포기김치 (김치:국내산)	1298
2016-02-23	화	2601	57	237	0	쌀밥/잡곡밥 (쌀:국내산) 콩나물국 돈육간장볶음 버섯잡채 꽃상추무침 요구르트 포기김치 (김치:국내산)	899
2016-02-24	수	2601	54	232	0	비빔밥 (쌀:국내산) 팽이장국 소세지피망볶음 맛탕 굴 요구르트 포기김치 (김치:국내산)	1022
2016-02-25	목	2601	56	261	0	쌀밥/잡곡밥 (쌀:국내산) 북어계란국 닭갈비 세쌍두부구이 곰피초장 요구르트 포기김치 (김치:국내산)	917
2016-02-26	금	2601	144	278	0	쌀밥/잡곡밥 (쌀:국내산) 수제비국 오징어볶음 계란찜 도라지초무침 요구르트 포기김치 (김치:국내산)	766
2016-02-29	월	2601	595	168	0	쌀밥/잡곡밥 (쌀,현미흑미:국내산) 열갈이된장국 제육볶음 늪은호박전 치커리생채 요구르트 포기김치 (배추,고추가루:국내산)	869

1. 데이터 분석

1.1. 데이터 개요

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

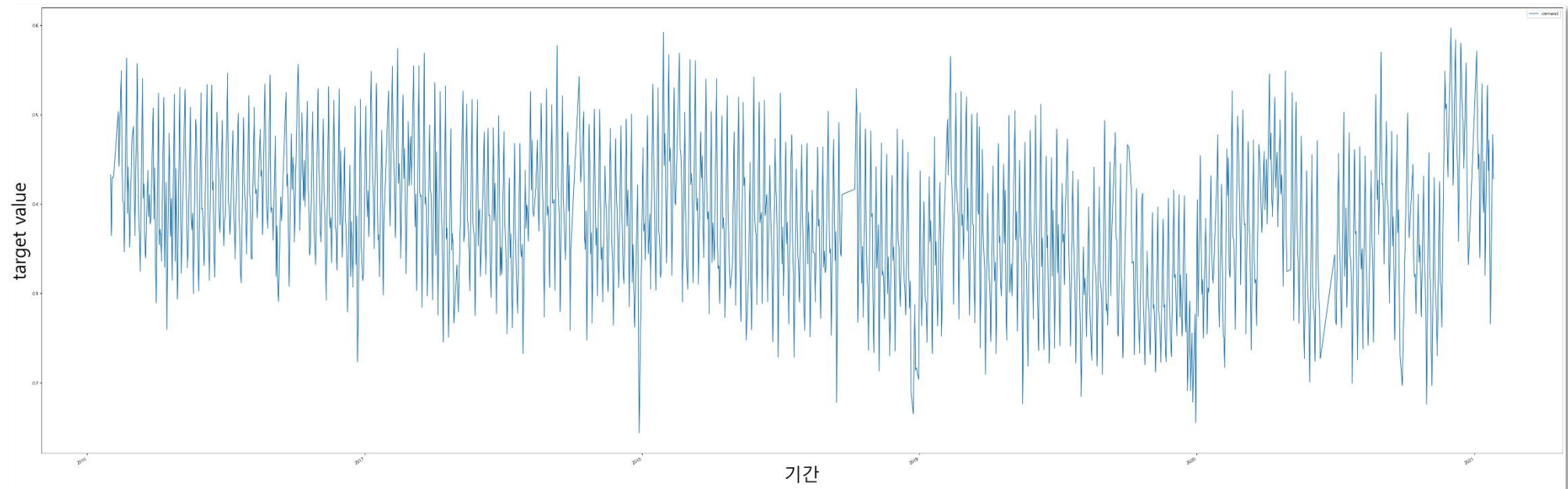
4. ML 모델

5. 하이브리드 모델

6. DL 모델

- Target value = 식수 인원/ 총 인원 수

* 총 인원 수 = 본사 정원 수 - 휴가자 수 - 출장자 수 - 재택근무자 수

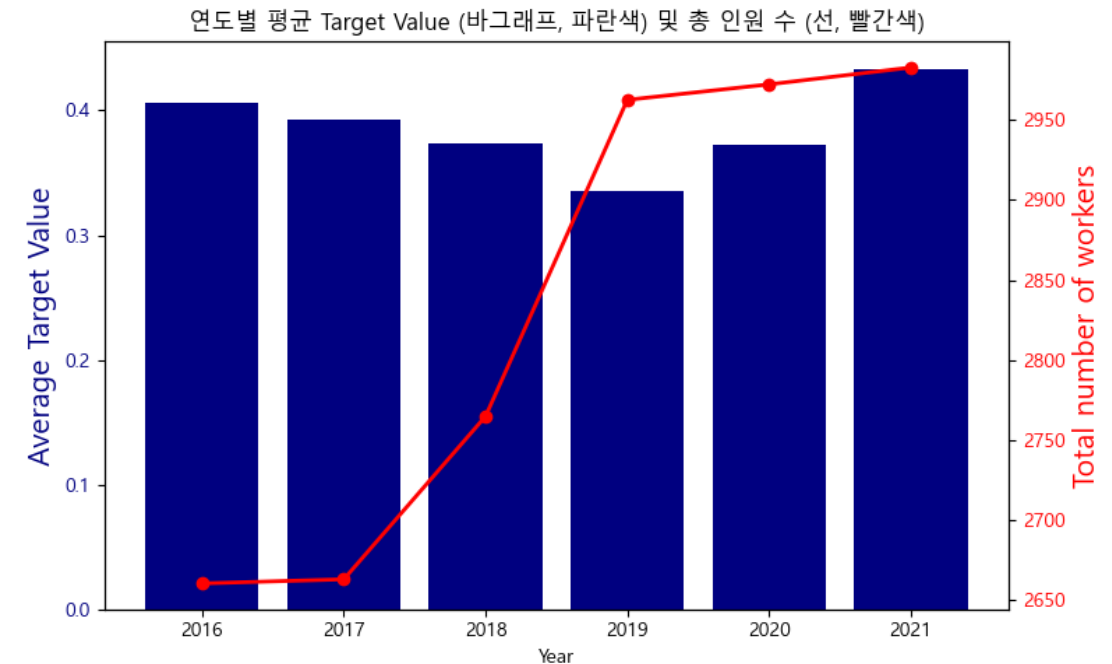
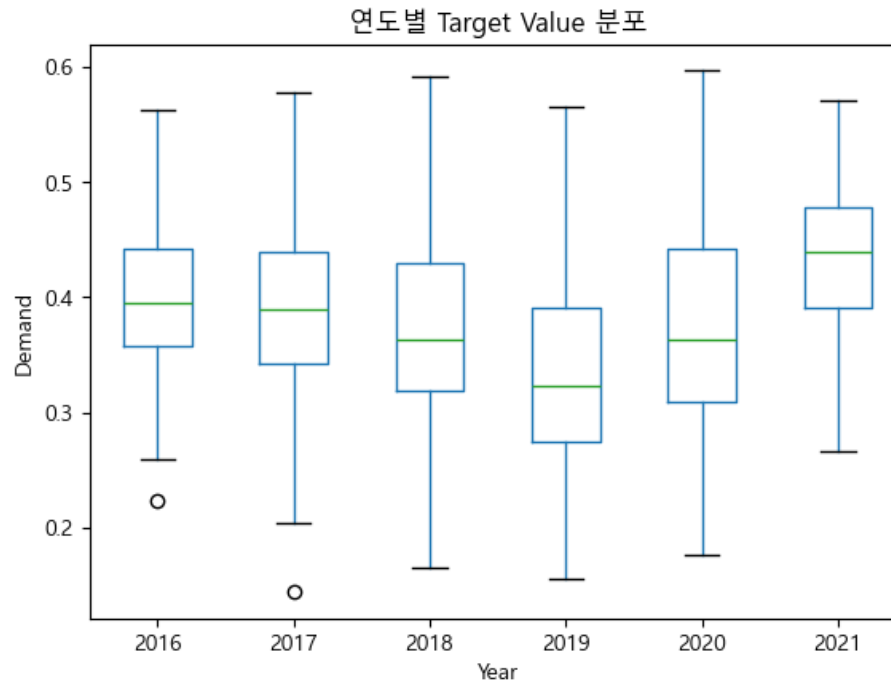


1. 데이터 분석

1.1. 데이터 개요

특성 1: 추세

- 2019년까지 수요가 감소하다가 이후부터 증가하는 추세를 확인할 수 있음

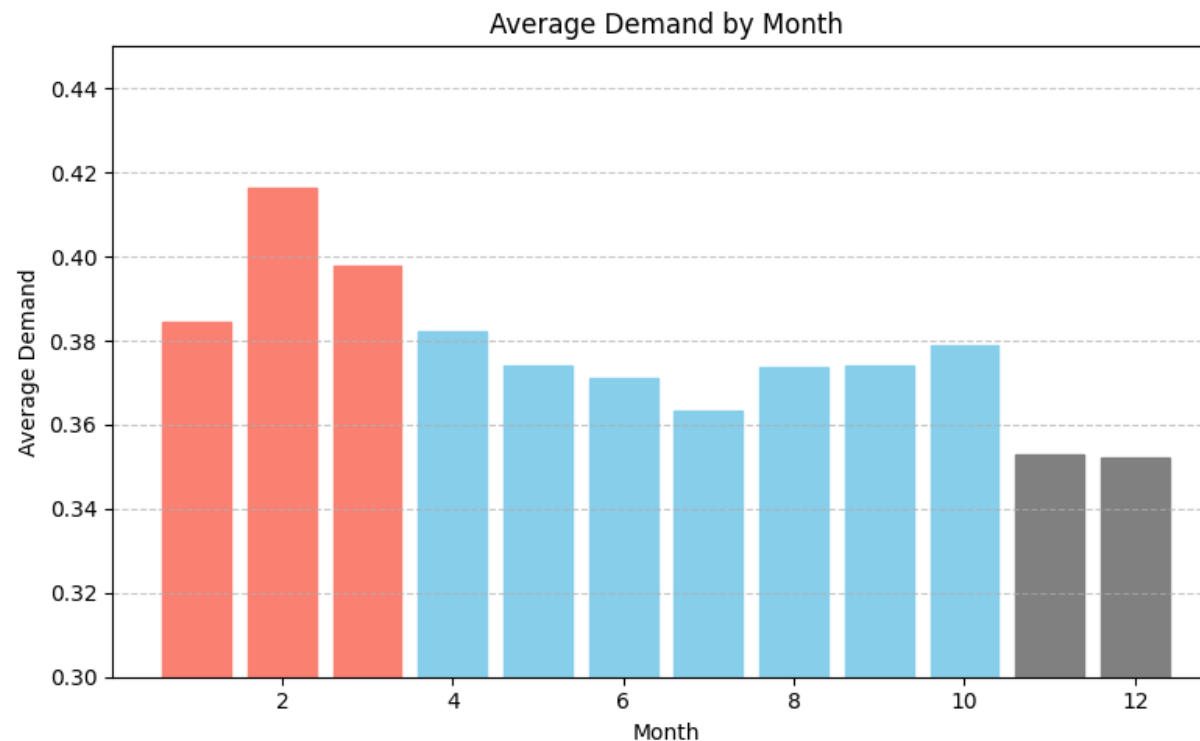


1. 데이터 분석

1.1. 데이터 개요

특성 2: 계절성 (주기: 월)

- 연초에 식수 인원이 많고 연말은 식수 인원이 적은 계절성 존재



0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

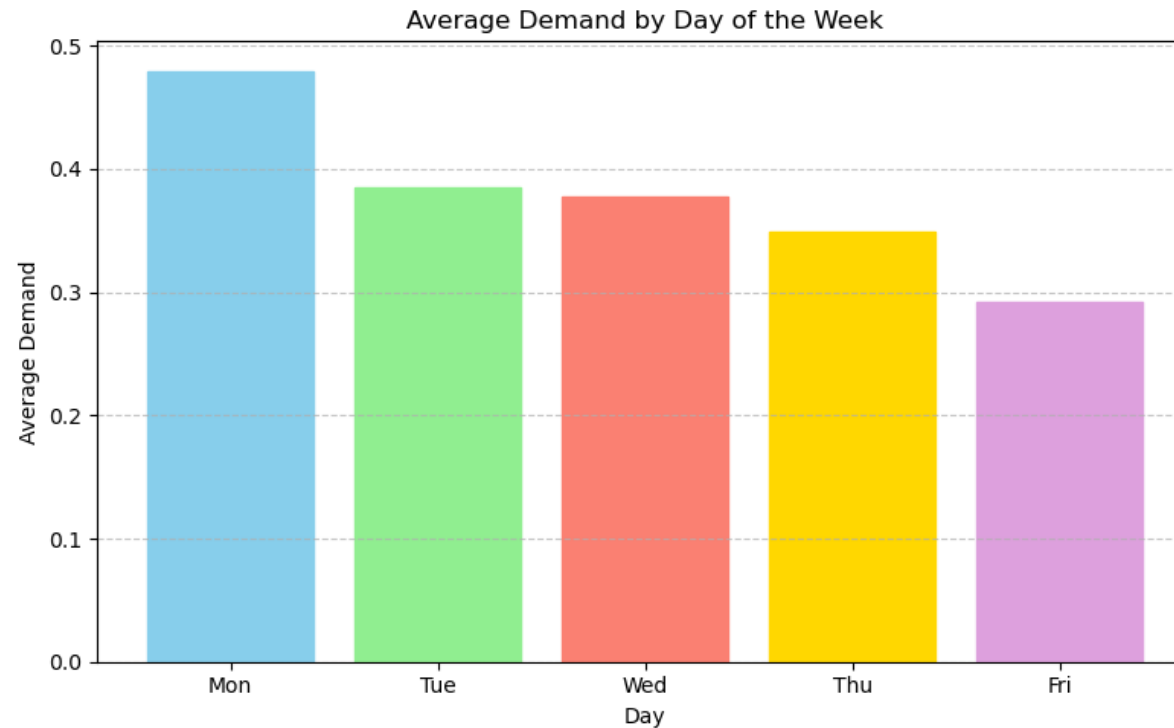
6. DL 모델

1. 데이터 분석

1.1. 데이터 개요

특성 2: 계절성 (주기: 일)

- 월요일에 식수 인원이 가장 많고 점차 식수 인원이 감소하는 계절성 존재



0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

1. 데이터 분석

1.2. 데이터 이해

예측 전 데이터 이해의 중요성

- ✓ 예측 모델 적용 전, 시계열 데이터의 기본 구조와 특성 파악이 필수
- ✓ 예측하고자 하는 시계열 데이터에 다음 요소가 있는지 확인:
 - 추세 / 계절성 / 불규칙성

데이터 이해가 모델 성능 결정

- ✓ 데이터의 구조를 모르고 모델을 적용하면 오적합/과소적합 발생 가능
- ✓ 정확한 모델 선택과 튜닝을 위하여,
- ✓ 데이터 특성에 대한 이해로부터 적절한 전처리 수행 및 변수 추가 필요

1. 데이터 분석

1.3. 데이터 특성

특성 3: 메뉴 지수 – 방법 A

- 메뉴에 대한 선호도가 식수 인원에게 영향을 미침을 예상할 수 있음
- 메뉴의 구성 요소를 다음의 4가지로 분류할 수 있음

분류	예시
돼지고기/소고기	갈비, 불고기, 떡갈비, 바비큐, 제육 등
닭/오리	삼계탕, 간풍기, 치킨, 오리 등
해산물	연어, 고등어, 낙지, 꽃게, 오징어 등
채소	두부, 부추, 파프리카, 콩나물 등

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

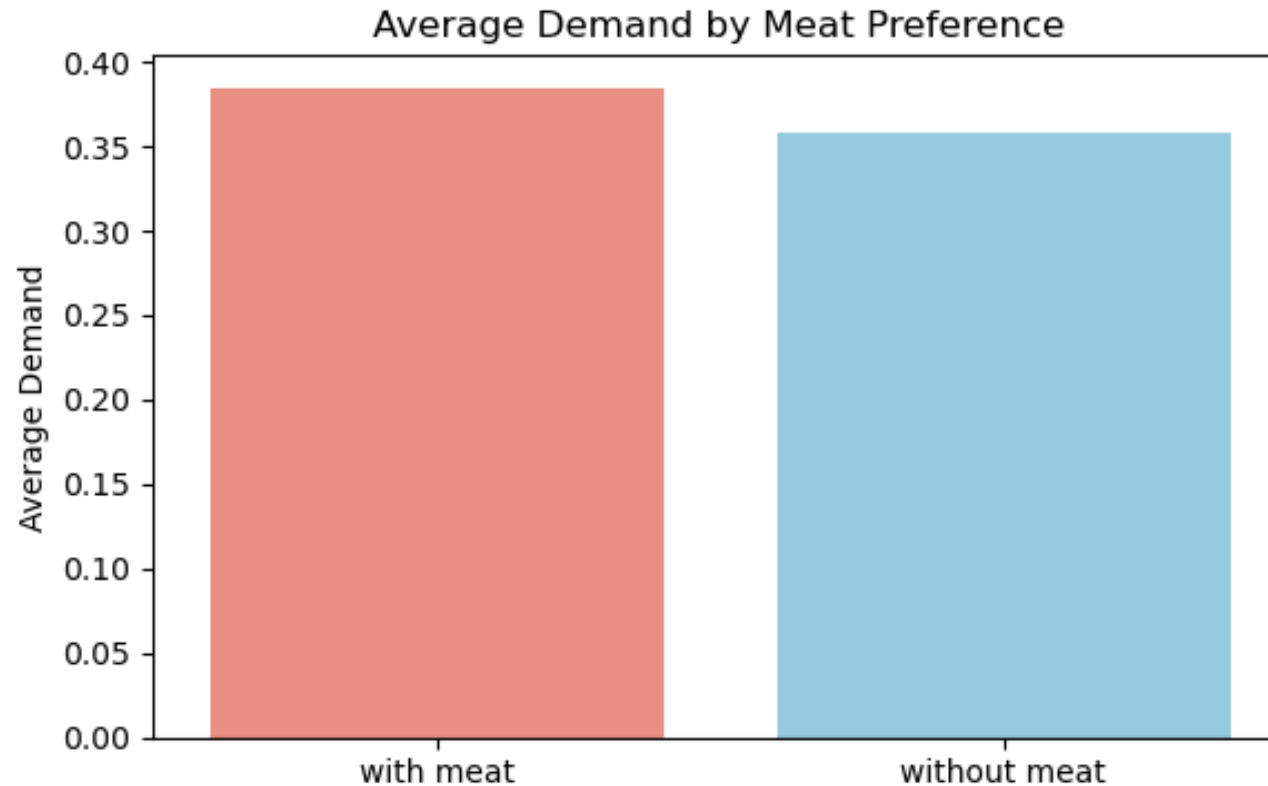
6. DL 모델

1. 데이터 분석

1.3. 데이터 특성

특성 3: 메뉴 지수 – 방법 A

- 소고기/돼지고기를 선호함 (닭/오리도 유사한 결과)



0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

1. 데이터 분석

1.3. 데이터 특성

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

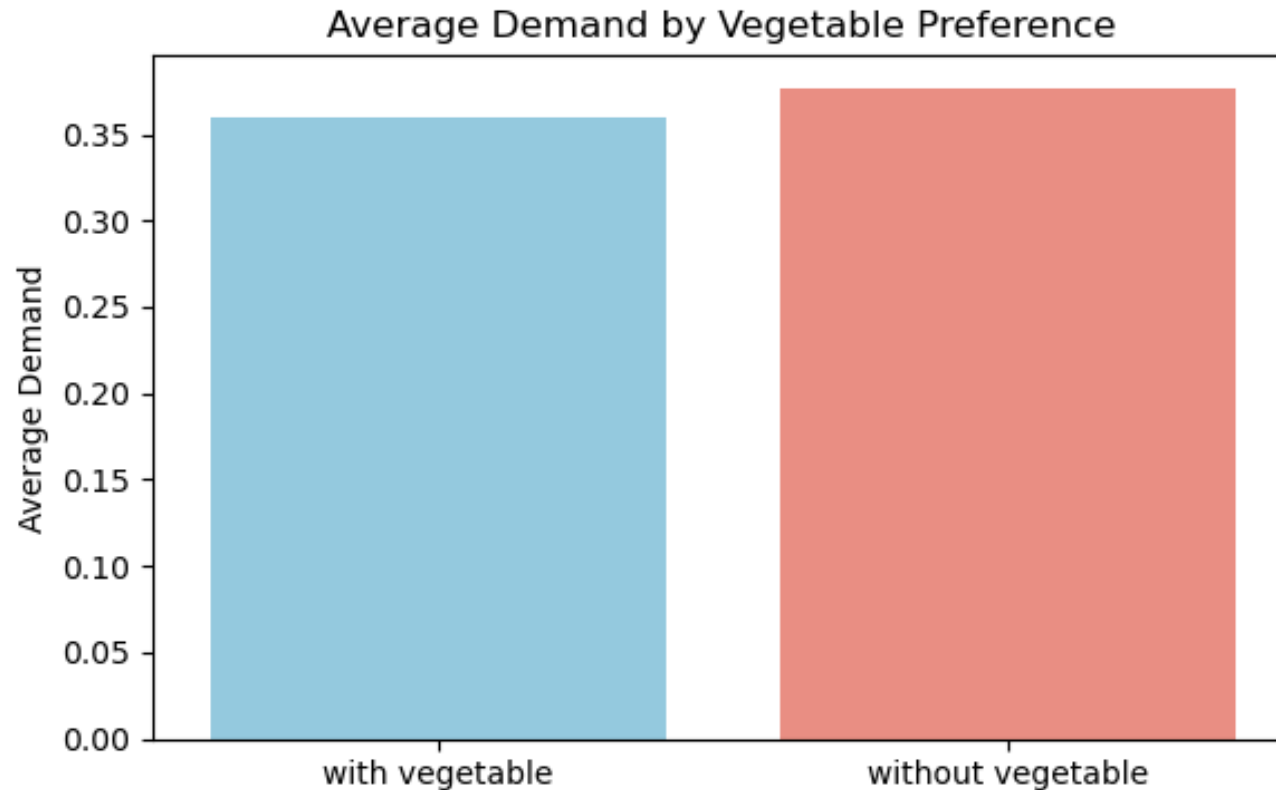
4. ML 모델

5. 하이브리드 모델

6. DL 모델

특성 3: 메뉴 지수 – 방법 A

- 채소를 선호하지 않음 (해산물도 유사한 결과)



1. 데이터 분석

1.3. 데이터 특성

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

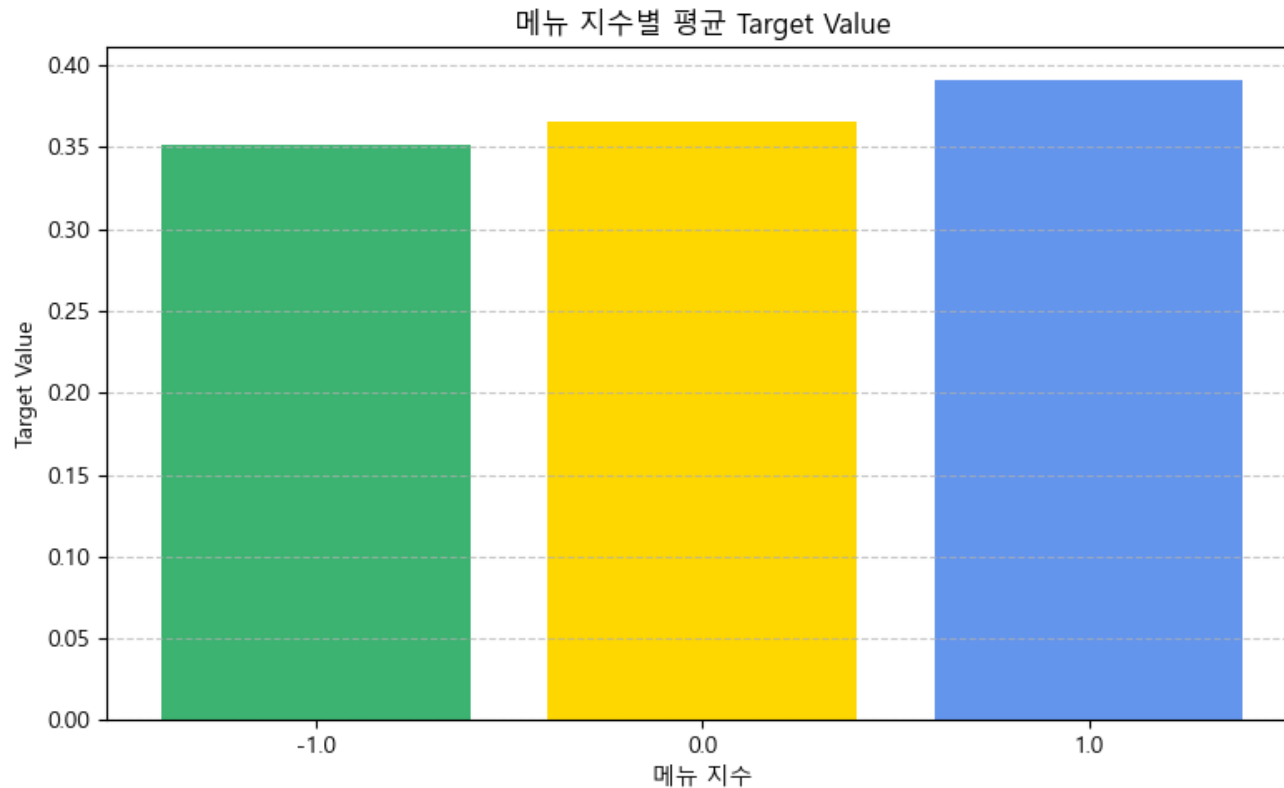
4. ML 모델

5. 하이브리드 모델

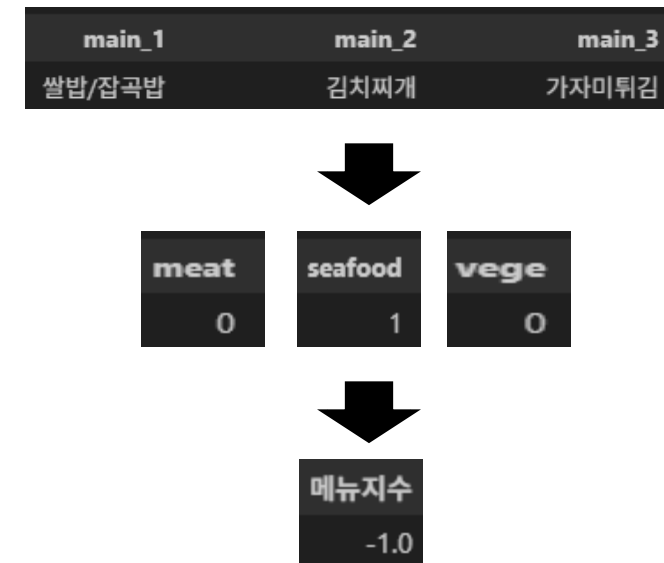
6. DL 모델

특성 3: 메뉴 지수 - 방법 A

- 메뉴 지수 = (육류 포함) - (해산물 포함) - (채소 포함)



예시



1. 데이터 분석

1.3. 데이터 특성

특성 3: 메뉴 지수 - 방법 B

- 중식 메뉴의 메인 메뉴 3가지 (밥 / 국 / 핵심 반찬) 추출
- 각 메인 메뉴가 나왔을 때의 식사 비율 평균으로 선호 지수(메뉴 지수) 산출

year	월	date	요일	실질인원	메인메뉴1	메인메뉴2	메인메뉴3	식사비율
2016	2	11	3	2075.0	쌀밥/잡곡밥	시래기국	훈제오리구이	0.503614
2016	3	18	4	2229.0	쌀밥/잡곡밥	시금치국	훈제오리구이	0.339614
2016	4	11	0	2358.0	쌀밥/잡곡밥	시래기국	훈제오리구이	0.519508
2016	5	9	0	2301.0	쌀밥/잡곡밥	매운어묵국	훈제오리구이	0.528466
2016	6	27	0	2314.0	쌀밥/잡곡밥	배추된장국	훈제오리구이	0.493518
2016	12	5	0	2397.0	쌀밥/잡곡밥	순두부찌개	훈제오리구이	0.463079
2017	1	18	2	2398.0	쌀밥/잡곡밥	아욱국	훈제오리구이	0.359883
2017	2	27	0	2317.0	쌀밥/잡곡밥	봄동된장국	훈제오리구이	0.492447
2017	6	7	2	2345.0	쌀밥/잡곡밥	콩나물국	훈제오리구이	0.480597
2017	9	1	4	2162.0	쌀밥/잡곡밥	시래기국	훈제오리구이	0.306660
2017	10	10	1	2331.0	쌀밥/잡곡밥	아욱국	훈제오리구이	0.542686
2017	12	21	3	2276.0	쌀밥/잡곡밥	순두부찌개	훈제오리구이	0.304042
2018	2	19	0	2281.0	쌀밥/잡곡밥	들깨미역국	훈제오리구이	0.569049
2018	4	17	1	2414.0	쌀밥/잡곡밥	무채국	훈제오리구이	0.372825
2018	5	14	0	2392.0	쌀밥/잡곡밥	시금치된장국	훈제오리구이	0.513796
2018	6	18	0	2412.0	쌀밥/잡곡밥	미역국	훈제오리구이	0.443201
2018	7	10	1	2357.0	쌀밥/잡곡밥	쇠고기매운국	훈제오리구이	0.355537

⋮

메뉴명	선호지수
가래떡돼지갈비찜	0.592144
삼겹살김치볶음	0.584035
누룽지장각백숙	0.580096
캔콩치튀김	0.580096
봉추찜닭	0.571138
...	...
훈제오리구이	0.417088
...	...
야채볶음밥*짜장소스	0.211965
해물수제비국	0.199380
수제탕수육	0.196911
감자양파국	0.191405
파인애플볶음밥	0.178423

전체 선호지수 평균

0.372071

1. 데이터 분석

1.3. 데이터 특성

외부 요인 및 파생 변수 활용 : 예측 정확도 향상에 기여

✓ 외부 요인 데이터 연계 가능성

- 문화의 날, 공휴일, 날씨 (기온, 강수량 등), 불쾌지수, 코로나 발생 추이 등

✓ 메뉴 선호도 반영

- 실습에서는 메인 메뉴 3가지에 대해 '선호 지수'를 산출

	year	월	date	요일	실질인원	메인메뉴1	메인메뉴2	메인메뉴3	메인메뉴1_선호지수	메인메뉴2_선호지수	메인메뉴3_선호지수	중식계
0	2016	2	1	0	2401.0	쌀밥/잡곡밥	오징어찌개	쇠불고기	0.379554	0.432736	0.435412	1039.0
1	2016	2	2	1	2378.0	쌀밥/잡곡밥	김치찌개	가자미튀김	0.379554	0.392490	0.325286	867.0
2	2016	2	3	2	2365.0	카레덮밥	팽이장국	치킨핑거	0.366182	0.374659	0.353322	1017.0
3	2016	2	4	3	2277.0	쌀밥/잡곡밥	쇠고기무국	주꾸미볶음	0.379554	0.390525	0.337742	978.0
4	2016	2	5	4	2142.0	쌀밥/잡곡밥	떡국	돈육씨앗강정	0.379554	0.395263	0.431839	925.0
...	...	...	...	...	...	...	...	...	...	...	...	...
1200	2021	1	20	2	2319.0	쌀밥/흑미밥/찰현미밥	아욱국	수제함박스테이크	0.393360	0.373795	0.471324	1093.0
1201	2021	1	21	3	2309.0	쌀밥/수수밥/찰현미밥	냉이된장국	동파육	0.372526	0.370020	0.399378	832.0
1202	2021	1	22	4	2177.0	전주비빔밥*약고추장	계란파국	요거달	0.265962	0.265962	0.265962	579.0
1203	2021	1	25	0	2396.0	쌀밥/흑미밥/찰현미밥	전주식콩나물해장국	돈육간장불고기	0.393360	0.477880	0.485043	1145.0
1204	2021	1	26	1	2369.0	쌀밥/귀리밥/찰현미밥	들깨미역국	교촌간장치킨	0.364855	0.374676	0.428451	1015.0

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

1. 데이터 분석

1.4. 파생변수 생성 실습

1. 데이터 전처리

1-1. 기존 변수 정리

```
data=pd.read_csv('./data/train.csv')
```

```
# 일자 관련 변수: 년도, 월, 일, 요일  
year=[]  
for i in range(len(data['일자'])) :  
    year.append(data.iloc[i,0][0:4])  
data['year']=year
```

```
mon=[]  
for i in range(len(data['일자'])) :  
    mon.append(data.iloc[i,0][5:7])  
data['월']=mon
```

```
date=[]  
for i in range(len(data['일자'])) :  
    date.append(data.iloc[i,0][8:10])  
data['date']=date
```

```
yoil=[]  
for i in range(len(data)) :  
    if data.iloc[i,1]=='월' : yoil.append(0)  
    if data.iloc[i,1]=='화' : yoil.append(1)  
    if data.iloc[i,1]=='수' : yoil.append(2)  
    if data.iloc[i,1]=='목' : yoil.append(3)  
    if data.iloc[i,1]=='금' : yoil.append(4)  
data['요일']=yoil
```

```
# 랜덤포레스트에서 categorical로 처리하기 위해 데이터 형변환  
data['date']=data['date'].astype(int)  
data['월']=data['월'].astype(int)  
data['year']=data['year'].astype(int)
```

```
#필요한 컬럼 선택
```

```
data=data[['일자', 'year', '월', 'date', '요일', '본사정원수', '본사휴가자수', '본사출장자수', '본사시간외근무명령서승인건수', '현본사소속재택근무자수', '중식메뉴', '중식계']]
```

✓ 0.1s

데이터 파일 불러오기

“일자” 열 전처리 : 2019-07-31 → 2019, 7, 31, 수요일

1. 데이터 분석

1.4. 파생변수 생성 실습

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

1-2. 새로운 변수 추가

- 문화의 날, 공휴일, 날씨, 불쾌지수, 코로나 등 유의미하다고 생각하는 데이터를 추가로 가져와서 활용할 수 있음.
- 본 실습에서는 메인 메뉴 3가지의 "선호 지수"를 산출하여 새로운 변수로 활용함.

```
# 메인메뉴 1,2,3 추출
def make_menu_token(data):
    main_menu_1 = []
    main_menu_2 = []
    main_menu_3 = []

    for idx in range(len(data)):
        # 중식메뉴 문자열을 공백 기준으로 나눈 뒤, 빈 항목 제거
        tmp = data.loc[idx, '중식메뉴'].split()

        cleaned_menu = []
        for menu in tmp:
            # 괄호 안 정보 제거
            menu_cleaned = re.sub(r'\(.*?\)', '', menu).strip()
            if menu_cleaned: # 공백이 아닌 경우에만 추가
                cleaned_menu.append(menu_cleaned)

        # 3개의 메인메뉴가 존재한다고 가정
        main_menu_1.append(cleaned_menu[0] if len(cleaned_menu) > 0 else '')
        main_menu_2.append(cleaned_menu[1] if len(cleaned_menu) > 1 else '')
        main_menu_3.append(cleaned_menu[2] if len(cleaned_menu) > 2 else '')

    data['메인메뉴1'] = main_menu_1
    data['메인메뉴2'] = main_menu_2
    data['메인메뉴3'] = main_menu_3
```

```
make_menu_token(data)
```

✓ 0.0s

'중식메뉴' 열에 담긴 긴 문자열 → 띄어쓰기 기준으로 분리

괄호 안의 정보 및 공백 제거

메인메뉴 1, 2, 3 추출

Python

1. 데이터 분석

1.4. 파생변수 생성 실습

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

```
##실질인원 산출 후 실질인원 대비 중식계의 비율을 식사비율로 정의
data['실질인원'] = data['본사정원수'] - data['본사휴가자수'] - data['본사출장자수'] - data['현본사소속재택근무자수']
data['식사비율'] = data['중식계']/data['실질인원']
```

✓ 0.0s

Python

식사비율(총 인원 대비 식사한 인원) 계산

```
# 데이터프레임: data
menu_scores = []

# 메뉴와 위치 정보 (컬럼 이름과 menu 위치)
menu_columns = ['메인메뉴1', '메인메뉴2', '메인메뉴3']
for col in menu_columns:
    tmp_df = data[[col, '식사비율']].copy()
    tmp_df.columns = ['중식메뉴', '식사비율']
    menu_scores.append(tmp_df)

# concat해서 메뉴별 demand 수집
all_menu_demand = pd.concat(menu_scores)

# 메뉴별 평균 demand 계산 (선호도)
menu_preference = all_menu_demand.groupby('중식메뉴').mean().sort_values(by='식사비율', ascending=False)
menu_preference = menu_preference.reset_index()
menu_preference.columns = ['메뉴명', '선호지수']
menu_preference
```

✓ 0.0s

Python

각 메인 메뉴의 선호 지수 (평균 식사비율) 도출

	메뉴명	선호지수
0	가래떡돼지갈비찜	0.592144
1	삼겹살김치볶음	0.584035
2	누룽지장각백숙	0.580096
3	캔공치튀김	0.580096
4	봉추찜닭	0.571138
...	...	...
725	야채볶음밥*파장소스	0.211965
726	해물수제비국	0.199380
727	수제탕수육	0.196911
728	감자양파국	0.191405
729	파인애플볶음밥	0.178423

1. 데이터 분석

1.4. 파생변수 생성 실습

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

```
# 선호지수를 원래 데이터에 매핑
## 메뉴-점수 매핑 딕셔너리 생성
score_map = dict(zip(menu_preference['메뉴명'], menu_preference['선호지수']))
```

```
## 점수 컬럼 추가
data['메인메뉴1_선호지수'] = data['메인메뉴1'].map(score_map)
data['메인메뉴2_선호지수'] = data['메인메뉴2'].map(score_map)
data['메인메뉴3_선호지수'] = data['메인메뉴3'].map(score_map)
```

```
data = data[['year', '월', 'date', '요일', '실질인원', '메인메뉴1', '메인메뉴2', '메인메뉴3', '메인메뉴1_선호지수', '메인메뉴2_선호지수', '메인메뉴3_선호지수', '종식계']]
data
```

✓ 0.0s

	year	월	date	요일	실질인원	메인메뉴1	메인메뉴2	메인메뉴3	메인메뉴1_선호지수	메인메뉴2_선호지수	메인메뉴3_선호지수	종식계
0	2016	2	1	0	2401.0	쌀밥/잡곡밥	오징어찌개	쇠불고기	0.379554	0.432736	0.435412	1039.0
1	2016	2	2	1	2378.0	쌀밥/잡곡밥	김치찌개	가자미튀김	0.379554	0.392490	0.325286	867.0
2	2016	2	3	2	2365.0	카레덮밥	팽이장국	치킨핑거	0.366182	0.374659	0.353322	1017.0
3	2016	2	4	3	2277.0	쌀밥/잡곡밥	쇠고기무국	주꾸미볶음	0.379554	0.390525	0.337742	978.0
4	2016	2	5	4	2142.0	쌀밥/잡곡밥	떡국	돈육씨앗강정	0.379554	0.395263	0.431839	925.0
...	...	...	...	...	...	...	...	...	...	...	...	...
1200	2021	1	20	2	2319.0	쌀밥/흑미밥/찰현미밥	아욱국	수제함박스테이크	0.393360	0.373795	0.471324	1093.0
1201	2021	1	21	3	2309.0	쌀밥/수수밥/찰현미밥	냉이된장국	동파육	0.372526	0.370020	0.399378	832.0
1202	2021	1	22	4	2177.0	전주비빔밥*약고추장	계란파국	요거달	0.265962	0.265962	0.265962	579.0
1203	2021	1	25	0	2396.0	쌀밥/흑미밥/찰현미밥	전주식콩나물해장국	돈육간장불고기	0.393360	0.477880	0.485043	1145.0
1204	2021	1	26	1	2369.0	쌀밥/귀리밥/찰현미밥	들깨미역국	교촌간장치킨	0.364855	0.374676	0.428451	1015.0

1205 rows × 12 columns

수요 예측 실습 개요

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

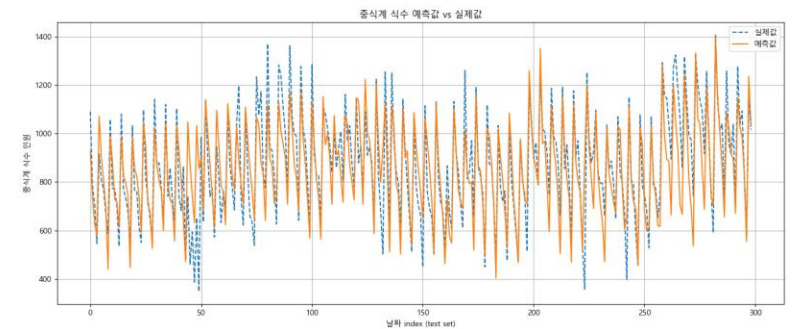
- 목표 : 2019년 10월 17일 이후 300일간의 수요를 예측하라!

- 모델 학습 및 성능 비교

- ✓ 회귀분석
- ✓ 시계열 예측 모델 (SARIMA)
- ✓ 머신러닝 모델
- ✓ 하이브리드 모델
- ✓ 딥러닝 모델 (참고)



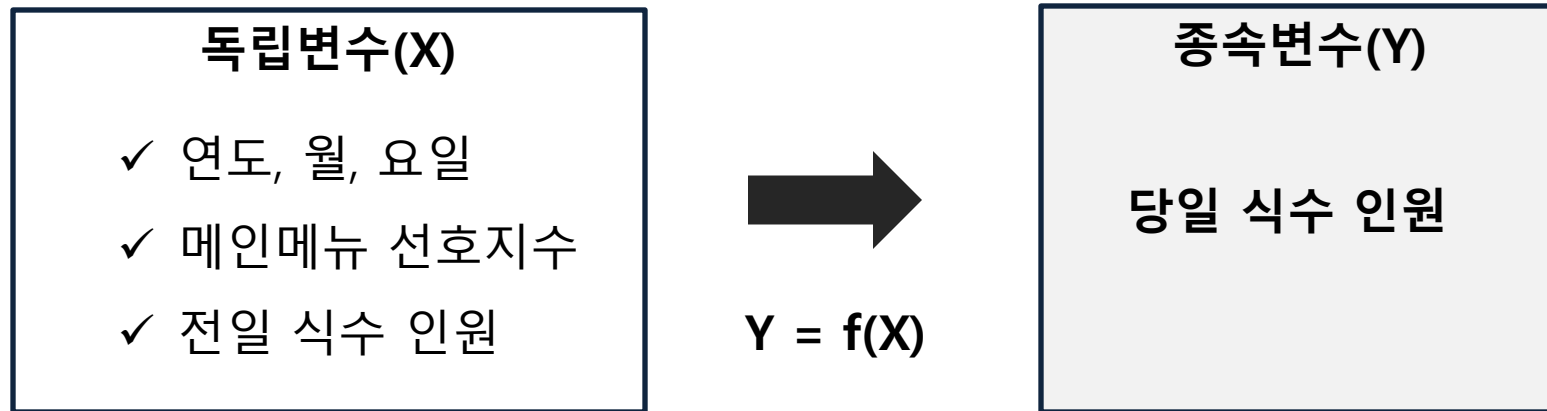
- 정량적 평가 : MAE, RMSE 등 오차 지표
- 선 그래프를 활용한 비교 시각화



2. 회귀분석

2.1. 회귀분석 복습

① 모델 설계 및 학습



- $y_t = \beta_k x_k^t + \dots + \beta_1 x_1^t + \beta_0 + \epsilon_t, t = 1, 2, \dots, N$
- Train data set에 대해 최소 제곱법을 이용해 최적의 β_k 를 학습함
- **statsmodels** library를 사용함

<https://www.statsmodels.org/stable/index.html>

2. 회귀분석

2.2. 회귀분석 실습

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

2. 회귀분석

2-1. 데이터 분할

```
# 학습(train) / 테스트(test) 세트 분리
train = data.iloc[:-300].copy()
test = data.iloc[-300:].copy()
✓ 0.0s
```

학습용 / 검증용 데이터 분할

Python

- 모델 학습에는 2016년 2월 1일부터 2019년 10월 17일까지의 데이터 사용
- 검증용으로는 2019년 10월 18일부터 2021년 1월 26일까지의 300일간 데이터를 별도로 분할하여 해당 구간의 식사 비율을 예측하도록 실습 진행

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

2. 회귀분석

2.2. 회귀분석 실습

2-2. 회귀모델 설계: 변수 선택 및 모델링

회귀 입력 변수 예시: 연, 월, 요일, 메인메뉴_선호지수, 전일 식수 인원(시차 변수)

```
# 필요한 열만 추출
features = ['year', '월', '요일', '메인메뉴1_선호지수', '메인메뉴2_선호지수', '메인메뉴3_선호지수']
target = '식사비율'
```

회귀모델에 넣을 변수 선정

```
# 시차 변수는 첫 번째 행에서 NaN 발생 → 제거
train = train.dropna()
test = test.dropna()
```

예측하고자 하는 목표 지정

```
# 모델 입력값 구성
X_train_raw = train[features]
X_test_raw = test[features]
y_train = train['식사비율']
y_test = test['식사비율']
```

데이터 스케일링

```
# 스케일링
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_raw)
X_test_scaled = scaler.transform(X_test_raw)
```

상수항 추가 (β_0 를 추정하기 위함)

```
# DataFrame으로 다시 변환
X_train = pd.DataFrame(X_train_scaled, columns=features, index=X_train_raw.index)
X_test = pd.DataFrame(X_test_scaled, columns=features, index=X_test_raw.index)

# 상수항 추가 (회귀분석의 질편을 위해)
X_train = sm.add_constant(X_train)
X_test = sm.add_constant(X_test)
```

✓ 0.0s

2. 회귀분석

2.2. 회귀분석 실습

2-3. 회귀 모델 학습 및 예측

```
# 회귀 모델 학습
model = sm.OLS(y_train, X_train)
results = model.fit()

# 결과 요약
print(results.summary())

# 예측
y_pred = results.predict(X_test)

✓ 0.0s
```

OLS Regression Results

Dep. Variable:	식사비용	R-squared:	0.695
Model:	OLS	Adj. R-squared:	0.693
Method:	Least Squares	F-statistic:	341.5
Date:	Thu, 05 Jun 2025	Prob (F-statistic):	9.12e-228
Time:	17:53:58	Log-Likelihood:	1523.3
No. Observations:	905	AIC:	-3033.
Df Residuals:	898	BIC:	-2999.
Df Model:	6		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	0.3813	0.002	254.217	0.000	0.378	0.384
year	-0.0197	0.002	-12.658	0.000	-0.023	-0.017
월	-0.0142	0.002	-9.285	0.000	-0.017	-0.011
요일	-0.0442	0.002	-24.908	0.000	-0.048	-0.041
메인메뉴1_선호지수	-0.0004	0.002	-0.244	0.807	-0.003	0.003
메인메뉴2_선호지수	0.0107	0.002	6.440	0.000	0.007	0.014
메인메뉴3_선호지수	0.0216	0.002	12.322	0.000	0.018	0.025

Omnibus:	58.665	Durbin-Watson:	1.598
Prob(Omnibus):	0.000	Jarque-Bera (JB):	183.309
Skew:	-0.256	Prob(JB):	1.57e-40
Kurtosis:	5.145	Cond. No.	1.94

R-squared 값

각 변수의 영향력 확인

일반적으로 0.05 이하면 해당 계수가 유의미하다고 판단

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

2. 회귀분석

2.2. 회귀분석 실습

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

2-4. 성능 평가

```
mae = mean_absolute_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"MAE (평균 절대 오차): {mae:.2f}")
print(f"RMSE (평균 제곱근 오차): {rmse:.2f}")
print(f"R2 (설명력): {r2:.4f}")
```

✓ 0.0s

```
MAE (평균 절대 오차): 0.05
RMSE (평균 제곱근 오차): 0.00
R2 (설명력): 0.5195
```

- 회귀 모델은 수요를 비율(0~1) 값으로 예측하는 문제이므로, 오차(MAE, RMSE) 역시 자연스럽게 작게 나타남

2. 회귀분석

2.2. 회귀분석 실습

② 수요 예측 결과

0. 실습 개요

1. 데이터 분석

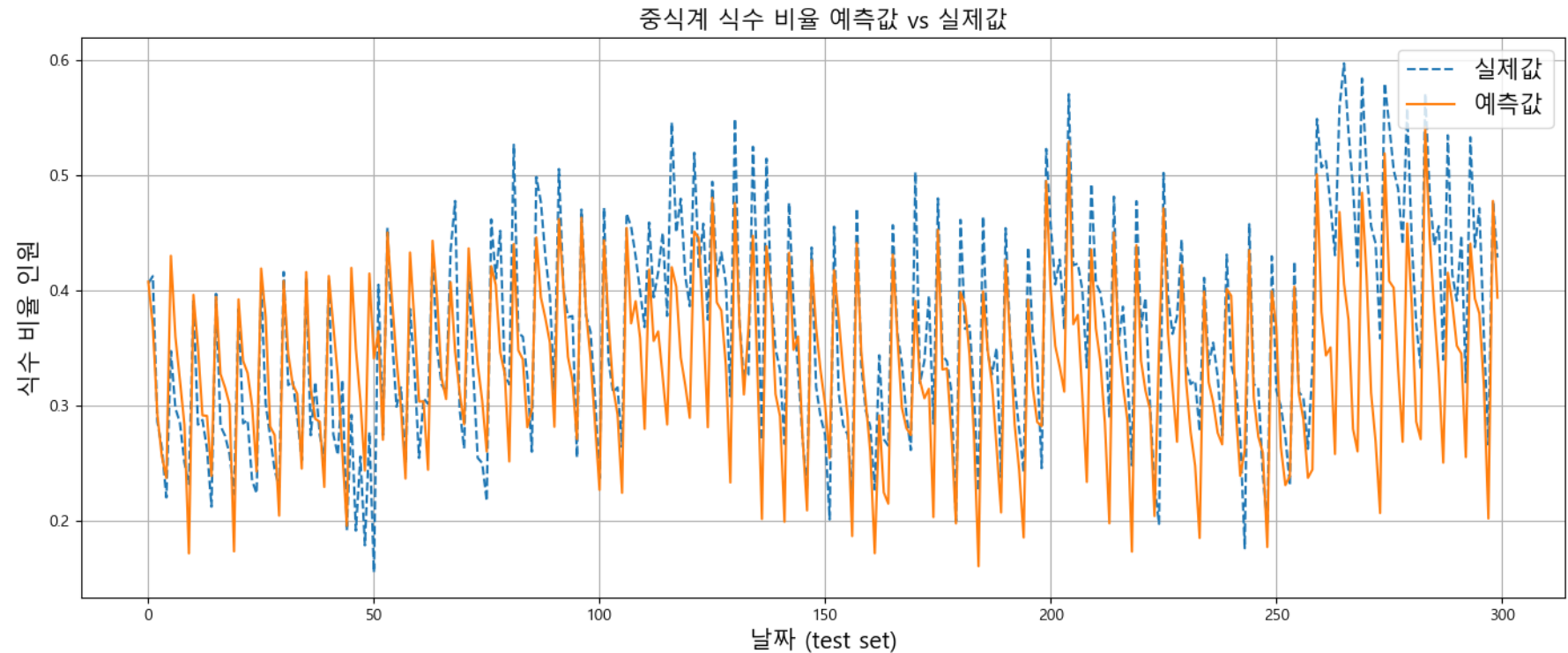
2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델



0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

3. 시계열 모델

3.1. 시계열 예측 모델 복습

① 모델 설계 및 학습

$$\underbrace{\phi(L)}_p \underbrace{(1-L)^d}_d X_t = \underbrace{\theta(L)}_q \varepsilon_t$$

AR (p) : 과거 p시점 관측치를 선형 결합하여 현재 값 예측

I (d) : 시계열을 d회 차분해 정상성 확보

MA (q) : 과거 q시점 오차를 이동 평균하여 잡음 제거

- 방법 1) Grid Search로 최적 파라미터 조합 탐색 (AIC 기준)
- 방법 2) 'pmdarima' 라이브러리를 통해 자동 탐색 가능
- 방법 1 혹은 2로 탐색한 best parameter 기반의 모델을 test set에 적용

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

3. 시계열 모델

3.2. 시계열 예측 모델 실습

3. 계절성과 추세를 반영하는 시계열 수요 예측 모델

- SARIMA
- Holt-Winters
- Prophet (conda terminal에서 conda install -c conda-forge prophet 명령어로 prophet 설치 필요)

3-1. 데이터 전처리 및 분할

```
# DataFrame 구성
data=pd.read_csv('./data/train.csv')

#필요한 컬럼 선택
data=data[['일자', '본사정원수', '본사휴가자수', '본사출장자수', '현본사소속재택근무자수', '중식계']]

##실질인원 산출 후 실질인원 대비 중식계의 비율을 식사비율로 정의
data['실질인원'] = data['본사정원수'] - data['본사휴가자수'] - data['본사출장자수'] - data['현본사소속재택근무자수']
data['식사비율'] = data['중식계']/data['실질인원']

data_ = data[['일자', '식사비율']]
data_.set_index('일자', inplace=True)

# 학습/테스트 분리
train = data_.iloc[:-50]
test = data_.iloc[-50:]
```

데이터 전처리

학습용 / 검증용 데이터 분할

✓ 0.0s

3. 시계열 모델

3.2. 시계열 예측 모델 실습

3-2. 파라미터 탐색

출력 결과

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

탐색 범위 지정

```
# 탐색할 파라미터 범위
p = d = q = range(0, 3)
P = D = Q = range(0, 2)
s = 5 # 주 단위 계절성
```

```
# 파라미터 조합 생성
pdq = list(itertools.product(p, d, q))
seasonal_pdq = list(itertools.product(P, D, Q))
```

```
# 결과 저장용
best_aic = np.inf
best_params = None
```

```
print("Grid Search 시작...")
```

Grid Search

```
for param in pdq:
    for seasonal_param in seasonal_pdq:
        try:
            mod = SARIMAX(train['식사비율'],
                           order=param,
                           seasonal_order=(seasonal_param[0], seasonal_param[1], seasonal_param[2], s),
                           enforce_stationarity=False,
                           enforce_invertibility=False)
            result = mod.fit(dispatch=False)

            if result.aic < best_aic:
                best_aic = result.aic
                best_params = (param, seasonal_param)
                print(f"Best AIC updated: {best_aic:.2f} | SARIMA{param}x{seasonal_param}")
        except:
            continue
```

```
print(f"\n 최적 파라미터: SARIMA{best_params[0]}x{best_params[1]} with AIC {best_aic:.2f}")
```

✓ 1m 23.1s

- Grid Search로 최적 파라미터 조합 탐색 (AIC 기준)
- 'pmdarima' 라이브러리를 통해 자동 탐색 가능

Grid Search 시작...

```
Best AIC updated: 1062.27 | SARIMA(0, 0, 0)x(0, 0, 0)
Best AIC updated: -165.06 | SARIMA(0, 0, 0)x(0, 0, 1)
Best AIC updated: -2814.49 | SARIMA(0, 0, 0)x(0, 1, 0)
Best AIC updated: -2946.27 | SARIMA(0, 0, 0)x(0, 1, 1)
Best AIC updated: -2960.19 | SARIMA(0, 0, 0)x(1, 0, 1)
Best AIC updated: -3010.98 | SARIMA(0, 0, 0)x(1, 1, 1)
Best AIC updated: -3011.48 | SARIMA(0, 0, 1)x(1, 1, 1)
Best AIC updated: -3047.93 | SARIMA(0, 1, 1)x(1, 0, 0)
Best AIC updated: -3061.20 | SARIMA(0, 1, 1)x(1, 0, 1)
Best AIC updated: -3062.08 | SARIMA(1, 0, 2)x(1, 0, 1)
```

최적 파라미터: SARIMA(1, 0, 2)x(1, 0, 1) with AIC -3062.08

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

3. 시계열 모델

3.2. 시계열 예측 모델 실습

3-3. 모델 학습 및 예측

```
# 최적 파라미터로 모델 재적합
model = SARIMAX(train['식사비율'],
                order=best_params[0],
                seasonal_order=(best_params[1][0], best_params[1][1], best_params[1][2], s))
results = model.fit(dispatch=False)
```

최적 파라미터를 활용하여 모델 학습

```
# 예측
start_idx = len(train)
end_idx = len(train) + len(test) - 1
forecast = results.predict(start=start_idx, end=end_idx)
forecast.index = test.index
```

식사비율 예측

✓ 0.5s

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

3. 시계열 모델

3.2. 시계열 예측 모델 실습

3-4. 성능 평가

```
# 평가
mae = mean_absolute_error(test['식사비율'], forecast)
rmse = sqrt(mean_squared_error(test['식사비율'], forecast))
r2 = r2_score(test['식사비율'], forecast)
```

```
print(f"MAE (평균 절대 오차): {mae:.2f}")
print(f"RMSE (평균 제곱근 오차): {rmse:.2f}")
print(f"R2 (설명력): {r2:.4f}")
```

✓ 0.0s

```
MAE (평균 절대 오차): 0.14
RMSE (평균 제곱근 오차): 0.17
R2 (설명력): -2.1787
```

- SARIMA는 시계열의 주간 패턴은 어느 정도 반영하고 있으나,
- 예측값이 일정한 형태로 반복되며 실제의 비선형적 변동성을 충분히 반영하지 못함

3. 시계열 모델

3.2. 시계열 예측 모델 실습

② 수요 예측 결과

0. 실습 개요

1. 데이터 분석

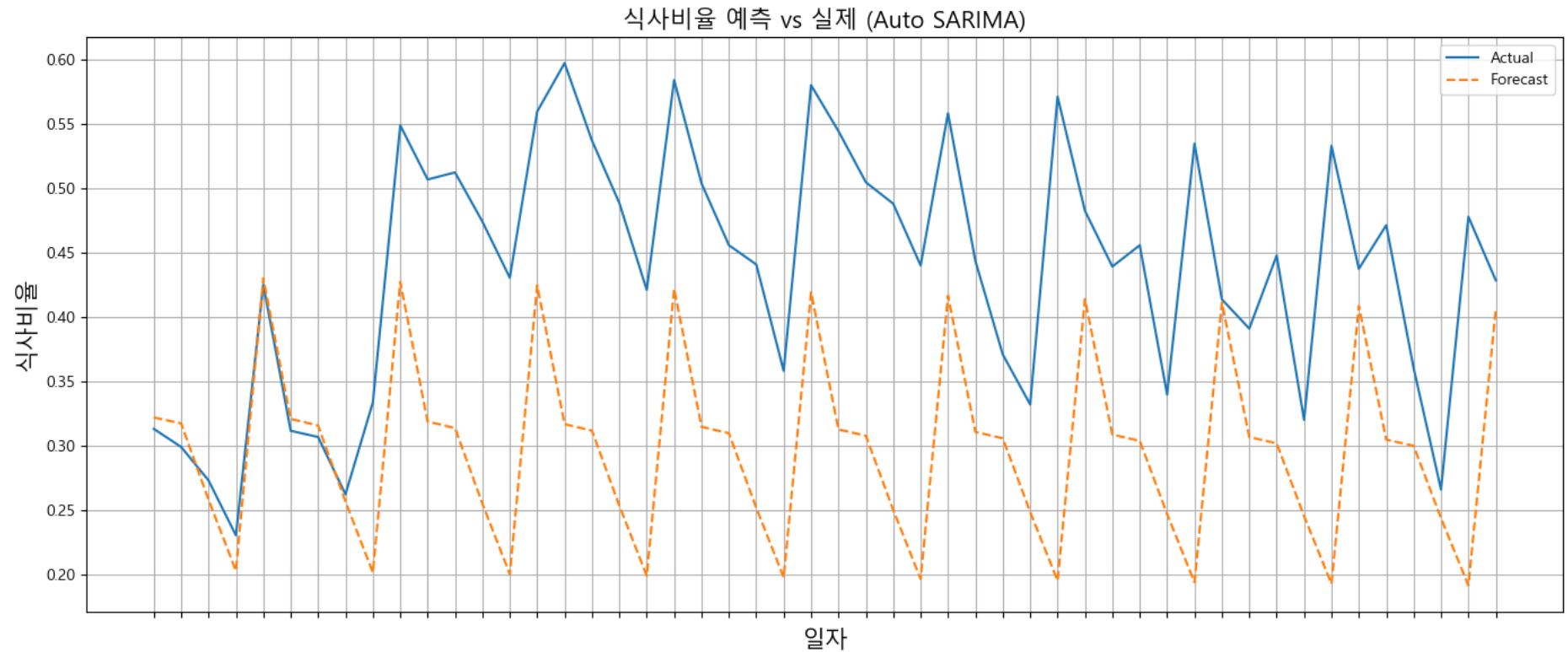
2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

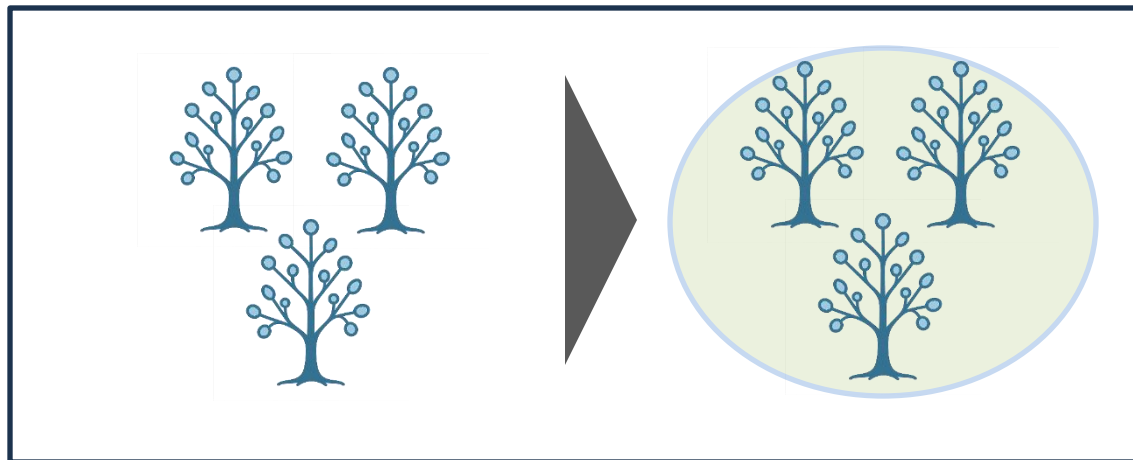


4. 머신러닝 모델

4.1. 머신러닝 모델 복습

① 모델 설계 및 학습

Random Forest 모델



- `n_estimators`, `max_depth` 등 다양한 파라미터 설정 필요
- `GridSearchCV`, `RandomizedSearchCV` 등으로 최적 파라미터 탐색 가능
- Best parameter로 학습 후 모델을 test set에 적용

Library 설명: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

4. 머신러닝 모델

4.2. 머신러닝 모델 실습

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

4-1. 데이터 전처리 및 분할

```
data=pd.read_csv('./data/train.csv')

#필요한 컬럼 선택
data=data[['일자', 'year', '월', 'date', '요일', '본사정원수', '본사휴가자수', '본사출장자수', '본사시간외근무명령서승인건수', '현본사소속재택근무자수', '중식메뉴', '중식계']]

make_menu_token(data)

##실질인원 산출 후 실질인원 대비 중식계의 비율을 식사비율로 정의
data['실질인원'] = data['본사정원수'] - data['본사휴가자수'] - data['본사출장자수'] - data['현본사소속재택근무자수']
data['식사비율'] = data['중식계']/data['실질인원']

# 선호지수를 원래 데이터에 매핑
## 메뉴-점수 매핑 딕셔너리 생성
score_map = dict(zip(menu_preference['메뉴명'], menu_preference['선호지수']))

## 점수 컬럼 추가
data['메인메뉴1_선호지수'] = data['메인메뉴1'].map(score_map)
data['메인메뉴2_선호지수'] = data['메인메뉴2'].map(score_map)
data['메인메뉴3_선호지수'] = data['메인메뉴3'].map(score_map)

data = data[['year', '월', 'date', '요일', '실질인원', '메인메뉴1_선호지수', '메인메뉴2_선호지수', '메인메뉴3_선호지수', '식사비율']]
```

데이터 전처리

✓ 0.1s

```
train = data.iloc[:-300,]
test = data.iloc[-300:,]
train_Y = train.loc[:, '식사비율']
train_X = train.drop(['식사비율'], axis=1)
test_X = test.drop(['식사비율'], axis=1)
```

✓ 0.0s

학습용 / 검증용 데이터 분할

4. 머신러닝 모델

4.2. 머신러닝 모델 실습

4-3. 모델 학습 및 예측

최적 파라미터 조합 탐색 생략

```
rf_model = RandomForestRegressor(random_state=535,n_estimators=110,max_depth=7)
rf_model.fit(train_X, train_Y)
```

```
rf_predict = rf_model.predict(test_X)
```

✓ 0.2s

4-4. 성능 평가

```
# 평가
mae = mean_absolute_error(test['식사비율'], rf_predict)
rmse = sqrt(mean_squared_error(test['식사비율'], rf_predict))
r2 = r2_score(test['식사비율'], rf_predict)
```

```
print(f"MAE (평균 절대 오차): {mae:.2f}")
print(f"RMSE (평균 제곱근 오차): {rmse:.2f}")
print(f"R² (설명력): {r2:.4f}")
```

✓ 0.0s

```
MAE (평균 절대 오차): 0.04
RMSE (평균 제곱근 오차): 0.06
R² (설명력): 0.6193
```

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

4. 머신러닝 모델

4.2. 머신러닝 모델 실습

② 수요 예측 결과

0. 실습 개요

1. 데이터 분석

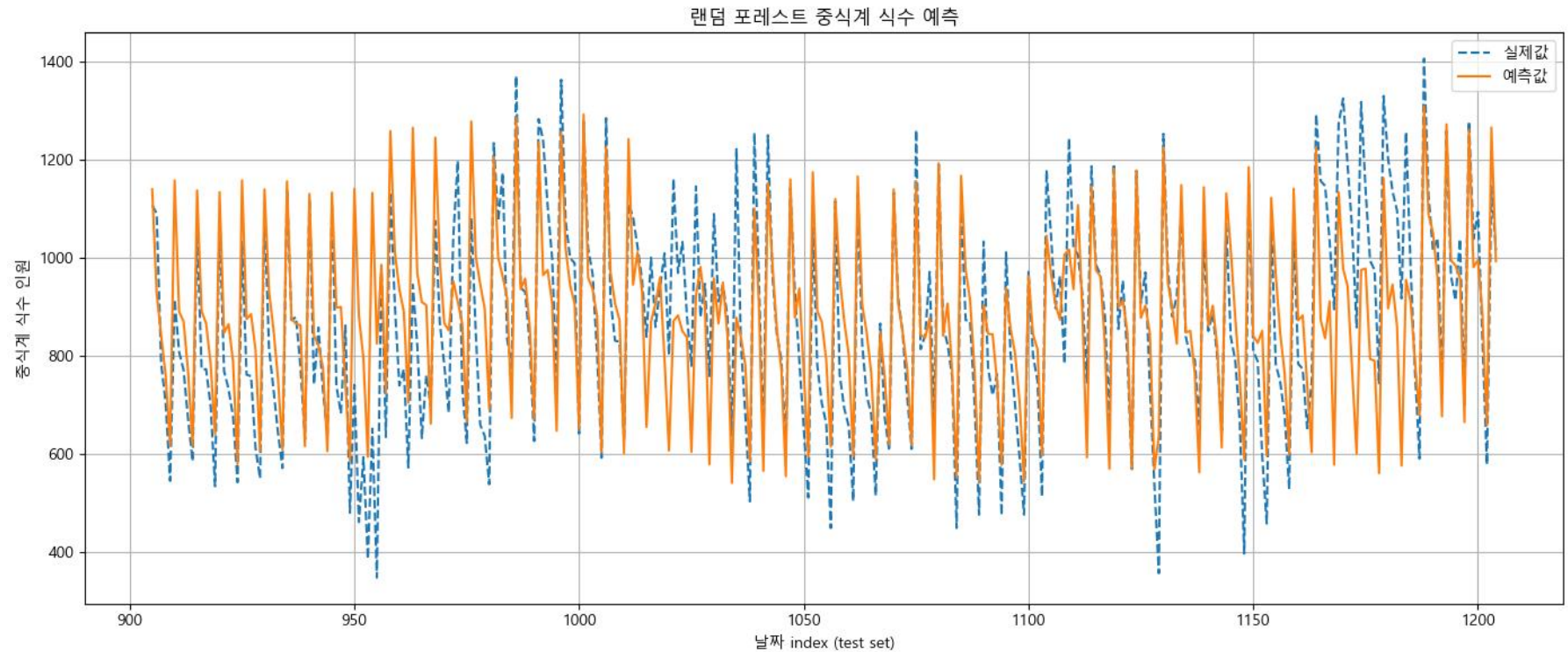
2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

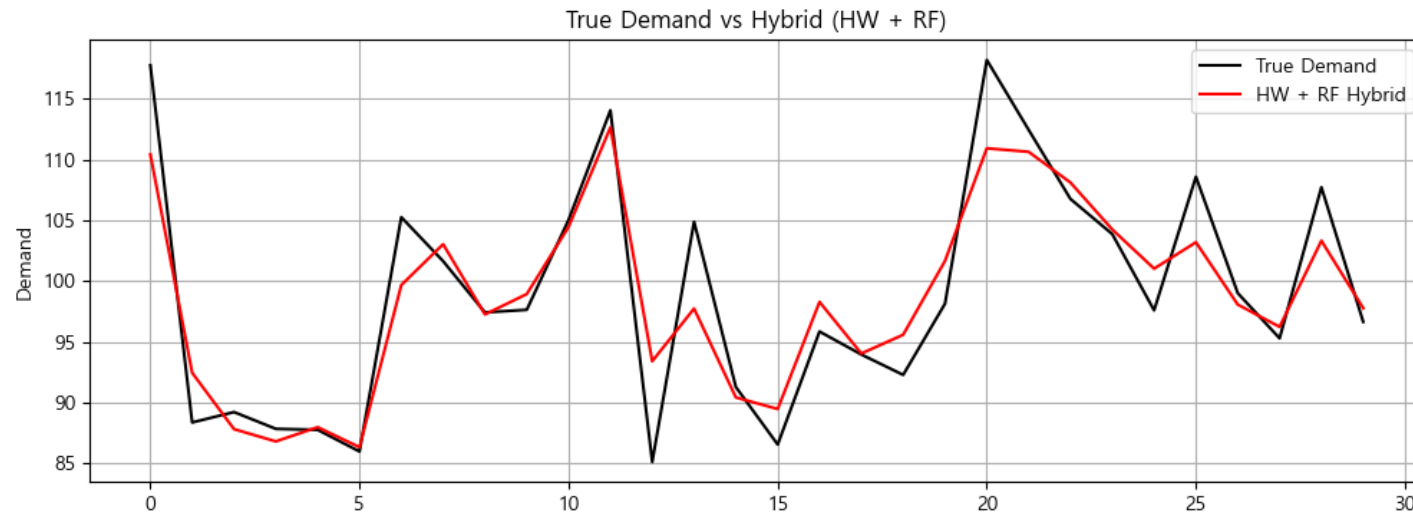
6. DL 모델



5. 하이브리드 모델

5.1. 하이브리드 모델 복습

① 모델 설계 및 학습 (Holt-Winters + Random Forest)



- Holt-Winters가 학습하지 못한 오차를 Random Forest로 학습시키는 방법
- Holt-Winters 단독 모델, Random Forest 단독 모델과 하이브리드 모델 비교

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

5. 하이브리드 모델

5.2. 하이브리드 모델 실습

5-1. Holt-Winters 예측

```
hw_model = ExponentialSmoothing(train_series, trend='add', seasonal='add', seasonal_periods=5).fit()
hw_forecast = hw_model.forecast(len(test_series))
mse_hw = mean_squared_error(test_series, hw_forecast)
```

5-2. Random Forest 단독 예측

```
X_train_rf = data[['실질인원', '메인메뉴1_선호지수', '메인메뉴2_선호지수', '메인메뉴3_선호지수']].iloc[:train_size]
y_train_rf = train_series
X_test_rf = data[['실질인원', '메인메뉴1_선호지수', '메인메뉴2_선호지수', '메인메뉴3_선호지수']].iloc[train_size:].reset_index(drop=True)
y_test_rf = test_series.reset_index(drop=True)

rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train_rf, y_train_rf)
rf_forecast = rf_model.predict(X_test_rf)
mse_rf = mean_squared_error(y_test_rf, rf_forecast)
```

5-3. Hybrid 예측 (HW + RF for residuals)

```
residuals = test_series.values - hw_forecast.values
y_resid = pd.Series(residuals).reset_index(drop=True)

rf_resid = RandomForestRegressor(n_estimators=100, random_state=42)
rf_resid.fit(X_test_rf, y_resid) # 보장은 테스트 구간 외생 변수에 대해
rf_resid_preds = rf_resid.predict(X_test_rf)

final_pred = hw_forecast.reset_index(drop=True) + rf_resid_preds
mse_hybrid = mean_squared_error(test_series, final_pred)
```

- 상세 코드 참조

5. 하이브리드 모델

5.2. 하이브리드 모델 실습

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

5-4. 성능 평가

```
print("📊 Mean Squared Error (Test Set):")
print(f"Holt-Winters only    : {mse_hw:.4f}")
print(f"Random Forest only   : {mse_rf:.4f}")
print(f"HW + RF (Hybrid)       : {mse_hybrid:.4f}")
```

✓ 0.4s

📊 Mean Squared Error (Test Set):

Holt-Winters only : 0.0135

Random Forest only : 0.0042

HW + RF (Hybrid) : 0.0009

※ R2 Score (Test Set):

Holt-Winters only : -0.5404

Random Forest only : 0.5240

HW + RF (Hybrid) : 0.9012

5. 하이브리드 모델

5.2. 하이브리드 모델 실습

② 수요 예측 결과

0. 실습 개요

1. 데이터 분석

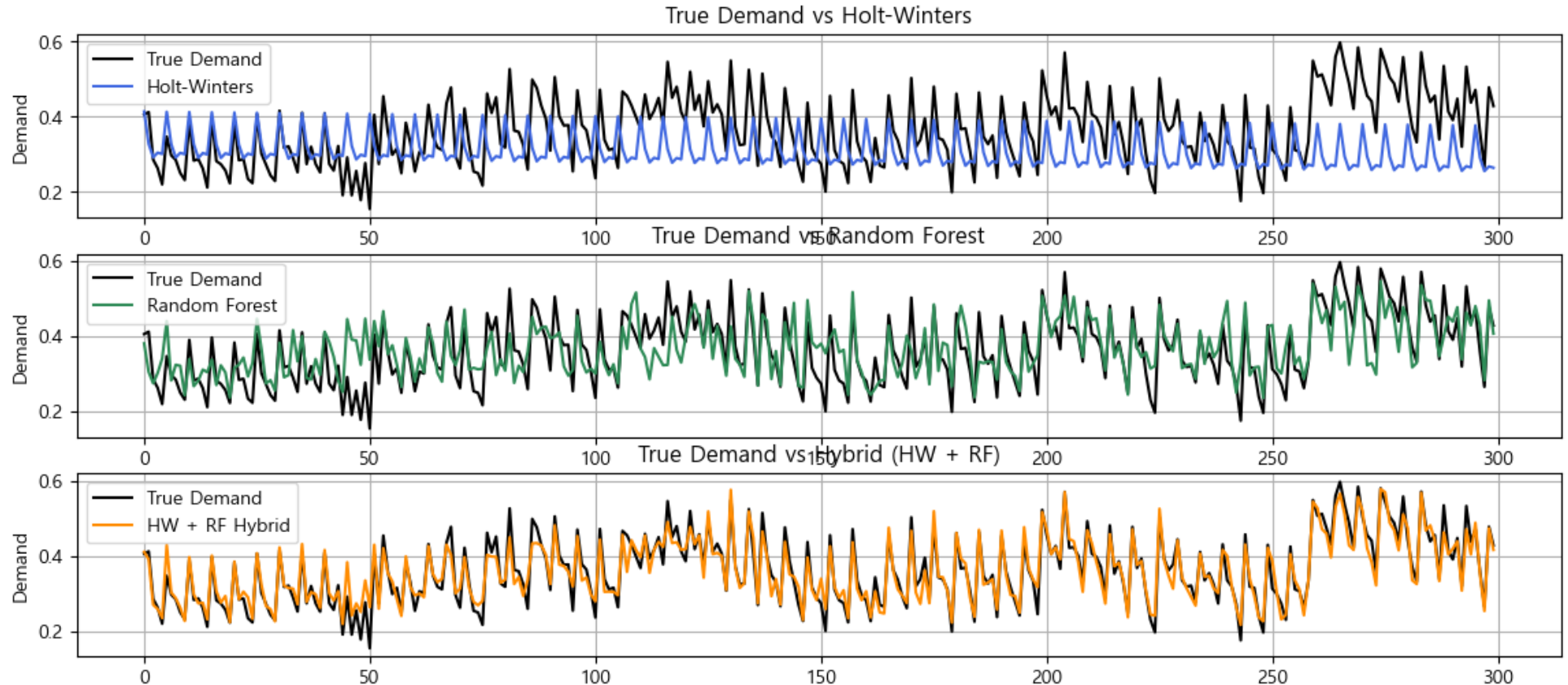
2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

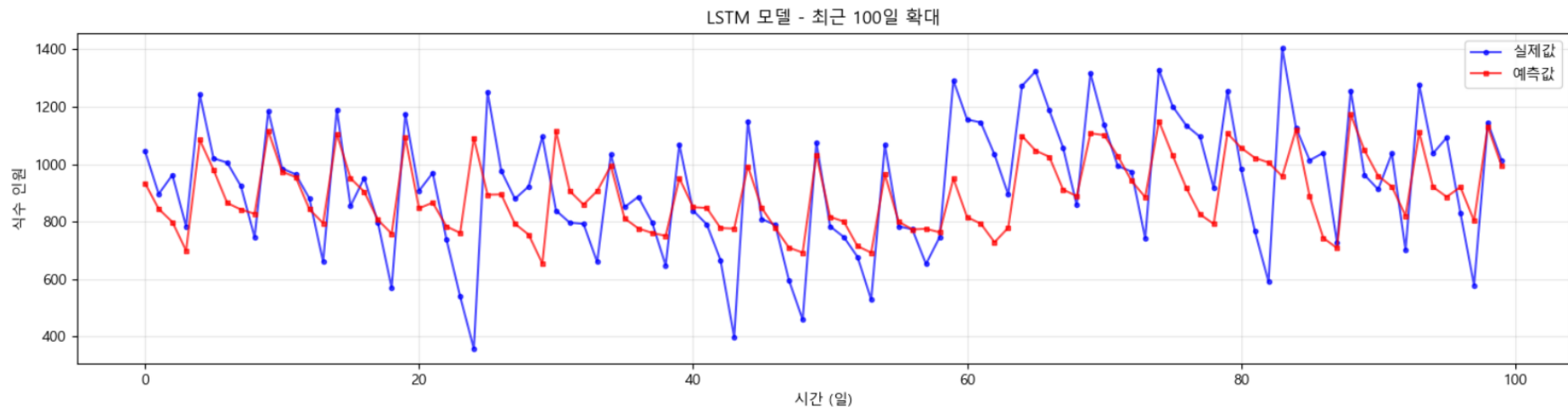
6. DL 모델



6. 딥러닝 모델 (참고)

6.1. 딥러닝 모델 복습

- ✓ 시계열 데이터의 흐름과 추세를 학습하는 데 강력한 기법 중 하나
- ✓ 과적합 방지, 최적 학습 시점 제어 필요
- ✓ 하이퍼파라미터 설정은 모델 성능에 중대한 영향
 - 노드 수, 모델 구조, optimizer 및 learning rate 등



0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

6. 딥러닝 모델 (참고)

6.2. 딥러닝 모델 실습

6. DL (참고)

```
def create_sequences(data, target_col, feature_cols, sequence_length=30):
    """
    시계열 데이터를 LSTM 입력 형태로 변환
    """
    X, y = [], []

    for i in range(sequence_length, len(data)):
        # 과거 sequence_length일 동안의 특성들
        X.append(data[feature_cols].iloc[i-sequence_length:i].values)
        # 예측할 타겟값
        y.append(data[target_col].iloc[i])

    return np.array(X), np.array(y)

def build_lstm_model(input_shape):
    """
    LSTM 모델 구성
    """
    model = Sequential([
        LSTM(64, return_sequences=True, input_shape=input_shape),
        Dropout(0.2),
        LSTM(32, return_sequences=False),
        Dropout(0.2),
        Dense(16, activation='relu'),
        Dense(1)
    ])

    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss='mse',
        metrics=['mae']
    )

    return model

def evaluate_model(y_true, y_pred):
    """
    모델 평가 지표 계산
    """
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)

    return mae, rmse, r2
```

출력 결과 (일부)

```
5. LSTM 모델 학습...
Epoch 1/100
23/23 ----- 2s 16ms/step - loss: 0.1897 - mae: 0.3817 - val_loss: 0.0518 - val_mae: 0.1882
Epoch 2/100
23/23 ----- 0s 5ms/step - loss: 0.0439 - mae: 0.1694 - val_loss: 0.0339 - val_mae: 0.1470
Epoch 3/100
23/23 ----- 0s 6ms/step - loss: 0.0345 - mae: 0.1493 - val_loss: 0.0350 - val_mae: 0.1486
Epoch 4/100
23/23 ----- 0s 5ms/step - loss: 0.0344 - mae: 0.1446 - val_loss: 0.0337 - val_mae: 0.1457
Epoch 5/100
23/23 ----- 0s 5ms/step - loss: 0.0331 - mae: 0.1459 - val_loss: 0.0334 - val_mae: 0.1453
Epoch 6/100
23/23 ----- 0s 5ms/step - loss: 0.0328 - mae: 0.1429 - val_loss: 0.0333 - val_mae: 0.1449
Epoch 7/100
23/23 ----- 0s 5ms/step - loss: 0.0312 - mae: 0.1407 - val_loss: 0.0332 - val_mae: 0.1445
Epoch 8/100
23/23 ----- 0s 5ms/step - loss: 0.0333 - mae: 0.1446 - val_loss: 0.0330 - val_mae: 0.1440
Epoch 9/100
23/23 ----- 0s 5ms/step - loss: 0.0308 - mae: 0.1384 - val_loss: 0.0331 - val_mae: 0.1440
Epoch 10/100
23/23 ----- 0s 5ms/step - loss: 0.0331 - mae: 0.1460 - val_loss: 0.0324 - val_mae: 0.1424
Epoch 11/100
23/23 ----- 0s 5ms/step - loss: 0.0328 - mae: 0.1471 - val_loss: 0.0336 - val_mae: 0.1455
Epoch 12/100
23/23 ----- 0s 6ms/step - loss: 0.0313 - mae: 0.1413 - val_loss: 0.0315 - val_mae: 0.1406
```

7. 모델 평가...

MAE (평균 절대 오차): 132.58
 RMSE (평균제곱근 오차): 173.65
 R² (결정계수): 0.3913

0. 실습 개요

1. 데이터 분석

2. 회귀분석

3. 시계열 모델

4. ML 모델

5. 하이브리드 모델

6. DL 모델

수요 예측 실습 정리

✓ 데이터 이해를 기반으로 모델 선정

- 시계열 데이터인지, 범주형 변수 포함 여부 등에 따라 적절한 알고리즘 선택

✓ 유의미한 요인선택 및 추가

- 계절성, 요일, 휴일, 날씨 등 예측에 영향을 줄 수 있는 변수 반영
- 도메인 지식을 바탕으로 feature engineering 수행

✓ 하이퍼파라미터 튜닝을 통한 성능 개선

- Grid Search, Random Search, Optuna 라이브러리 등 활용

✓ 하이브리드 또는 앙상블 모델 고려

- 서로 다른 예측 모델의 결과를 조합하여 예측 정확도 향상
- 최근 우수한 성능을 지속적으로 입증

Thank you for your attention.



Professor ILKYEONG MOON
Department of Industrial Engineering
Seoul National University
Seoul 08826, KOREA
ikmoon@snu.ac.kr
<http://scm.snu.ac.kr>